DİL MODELLERİNDE İNCE AYAR REHBERİ

Sohbet Şablonları (Chat Templates), Denetimli İnce Ayar (SFT), LoRA ve Değerlendirme Metodolojileri

EĞİTİM KILAVUZU HAKKINDA:
Bu doküman, Hugging Face ekosistemi üzerinde üretken yapay zeka ve büyük dil modellerinin (LLM) özelleştirilmesi, talimat takip yeteneklerinin kazandırılması, verimli ince ayar yöntemleri (PEFT/LoRA) ve sistematik değerlendirme süreçlerini içeren 6 temel kaynağın tam metin Türkçe çevirisini ve kapsamlı teknik sentezini sunmaktadır. Kod örnekleri, parametre tabloları, sohbet şablonu yapıları ve sınav soruları eksiksiz olarak aktarılmıştır.

Yayınlayan: Hugging Face / TRL & PEFT Ekosistemi Kılavuzu
Dil: Türkçe (Tam Metin Çeviri ve Birleştirilmiş Rehber)
Kapsam: 1-Supervised Fine-Tuning, 2-Chat Templates, 3-SFT Practical, 4-LoRA, 5-Evaluation, 6-Conclusion

1. Bölüm: Denetimli İnce Ayar (Supervised Fine-Tuning - SFT) Genel Bakış

Gelişmiş yapay zeka uygulamalarında, önceden eğitilmiş temel dil modellerini (Base Models) belirli iş kollarına veya insan tercihlerine uyarlamanın ana yolu **Denetimli İnce Ayar (Supervised Fine-Tuning - SFT)** işlemidir. SFT, ham metin üzerinde bir sonraki jetonu (token) tahmin etmek üzere eğitilmiş temel modelleri, kullanıcı talimatlarına uygun yanıtlar veren yardımsever dijital asistanlara dönüştürmektedir.

Temel Modeller ve Talimat Modelleri Arasındaki Farklar

Temel Modeller (Base Models): İnternet ölçeğindeki devasa metin verileri üzerinde eğitilirler. Temel amaçları verilen girdinin devamını istatistiksel olarak tamamlamaktır. Örneğin 'Fransa'nın başkenti' cümlesini verdiginizde 'neresidir?' veya 'hakkında bilgi' gibi devam cümleleri üretebilirler; doğrudan soruyu yanıtlamayabilirler.

Talimat / Sohbet Modelleri (Instruction / Chat Models): SFT sürecinden geçirilmiş modellerdir. Kullanıcının sorduğu 'Fransa'nın başkenti neresidir?' sorusuna doğrudan 'Fransa'nın başkenti Paris'tir.' şeklinde hedefe yönelik yanıt verirler.

SFT ile İstem Mühendisliği (Prompt Engineering) Karşılaştırması

Bir dil modelinden istenen davranışı elde etmek için ilk adım genellikle İstem Mühendisliğidir. Ancak karmaşık sistemlerde SFT kaçınılmaz hale gelir. Aşağıdaki tabloda bu iki yaklaşımın teknik karşılaştırması sunulmuştur:

Kriter	İstem Mühendisliği (Prompt Engineering)	Denetimli İnce Ayar (SFT)
Eğitim İhtiyacı	Model ağırlıkları değiştirilmez.	Model ağırlıkları güncellenir.
Donanım / Kaynak	Sıfır GPU eğitimi gerektirir.	GPU bellek ve hesaplama gücü gerektirir.
Çıkarım Maliyeti	Girdi bağlamı uzun olduğundan jeton maliyeti yüksektir.	Uzun talimatlar model ağırlıklarına işlendiğinden jeton tasarrufu sağlar.
Format Çıktı Garantisi	Özellikle küçük modellerde strictly JSON/XML çıktısı garanti edilemez.	Model belirli çıktı formatlarını katı bir şekilde öğrenir.
Özel Alan Bilgisi	Bağlam penceresi sınırlarına takılır.	Milyonlarca alan verisi model hafızasına kazınır.

2. Bölüm: Sohbet Şablonları (Chat Templates) ve Yapılandırma

Sohbet modellerinin sorunsuz çalışabilmesi, çoklu tur (multi-turn) konuşmalarında kullanıcı, sistem ve asistan rollerinin birbirine karışmaması için **Sohbet Şablonları (Chat Templates)** kullanılır. Sohbet şablonları, farklı model mimarilerinin ihtiyaç duyduğu özel jetonları (special tokens) ve mesaj sınırlarını standart bir yapıda birleştirir.

Sohbet Rolleri (Roles in Dialogue)

Bir sohbet yapısında üç temel rol tanımlanır:

• **System (Sistem):** Modelin genel kişiliğini, uyacağı kuralları, güvenlik sınırlarını ve yanıt formatını belirler.

• **User (Kullanıcı):** Modelle etkileşime giren insanın girdiği soru, talimat veya komutlardır.

• **Assistant (Asistan):** Modelin ürettiği yanıtlar veya geçmiş sohbetlerdeki asistan mesajlarıdır.

Yaygın Sohbet Formatları ve Özel Jetonlar

Farklı yapay zeka laboratuvarları kendi özel sohbet formatlarını geliştirmiştir. En yaygın formatlar aşağıda açıklanmıştır:

Format / Model	Açılış ve Kapanış Jetonları	Yapısal Örnek
ChatML (OpenAI / Qwen)	<|im_start|> role \n ... <|im_end|>	<|im_start|>user\nMerhaba<|im_end|>\n<|im_start|>assistant\n
Llama 2	[INST] <> ... <> ... [/INST]	[INST] <>Kurallar<> Merhaba [/INST]
Llama 3	<|start_header_id|>role<|end_header_id|>\n\n...<|eot_id|>	<|start_header_id|>user<|end_header_id|>\n\nMerhaba<|eot_id|>
Mistral / Mixtral	[INST] ... [/INST] ...	[INST] Soru [/INST] Yanıt

AutoTokenizer.apply_chat_template Kullanımı

Hugging Face `transformers` kütüphanesinde `AutoTokenizer`, modellerin `tokenizer_config.json` dosyasında yer alan Jinja2 şablonlarını otomatik olarak uygular. `apply_chat_template` fonksiyonu ile mesaj listeleri doğrudan modelin anlayacağı ham metne dönüştürülür:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained('HuggingFaceTB/SmolLM2-360M-Instruct')

messages = [
    {'role': 'system', 'content': 'Sen yardımsever ve nazik bir yapay zeka asistanısın.'},
    {'role': 'user', 'content': 'Kuantum bilgisayar nedir?'}
]

# Metin çıktısı almak için (tokenize=False)
prompt = tokenizer.apply_chat_template(messages, tokenize=False, add_generation_prompt=True)
print(prompt)
# Çıktı: &lt;|im_start|&gt;system\nSen yardımsever...&lt;|im_end|&gt;\n&lt;|im_start|&gt;user\nKuantum...&lt;|im_e
```

Gelişmiş Özellikler: Araç Kullanımı (Tool Use) ve Görselli İstemler

Modern sohbet şablonları yalnızca metinleri değil, harici fonksiyon çağırma (Function Calling / Tool Use) ve görsel-dil modelleri (VLM) için çok modlu (multimodal) girdileri de destekler. Araç kullanımında model, kullanılabilecek Python fonksiyonlarının JSON şemalarını alır ve uygun durumlarda yanıt yerine fonksiyon çağırma yapısını üretir.

3. Bölüm: Denetimli İnce Ayar Uygulaması (SFT with TRL)

Hugging Face `trl` (Transformer Reinforcement Learning) kütüphanesi, SFT süreçlerini optimize eden **SFTTrainer** ve **SFTConfig** sınıflarını sunar. Bu araçlar veri kümesi paketleme (packing), dizi uzunluğu kırpma ve paralel eğitimi otomatik olarak yönetir.

SFTConfig Yapılandırma Parametreleri

Eğitim sürecini kontrol eden kritik `SFTConfig` parametreleri aşağıda özetlenmiştir:

Parametre	Tip / Varsayılan	Açıklama ve En İyi Kullanım Pratiği
max_seq_length	int (ör. 512 / 2048)	Modelin kabul edeceği maksimum dizi uzunluğudur. Bellek kullanımını doğrudan etkiler.
dataset_text_field	str ('text')	Veri kümesindeki metin sütununun adıdır.
packing	bool (True / False)	Kısa örnekleri birleştirerek diziyi max_seq_length boyutuna doldurur. GPU verimini %30-50 artırır.
per_device_train_batch_size	int (ör. 4 / 8)	Her GPU cihazı başına düşen miniank yükü.
gradient_accumulation_steps	int (ör. 2 / 4)	Bellek yetersizliğinde efektif batch boyutunu artırmak için gradyan biriktirme adımı.
learning_rate	float (ör. 2e-5)	Öğrenme oranı. SFT için genellikle 1e-5 ile 5e-5 arasında seçilir.
num_train_epochs	int (ör. 3)	Veri kümesi üzerinden kaç tam geçiş yapılacağı.

Uygulamalı SFT Eğitim Kodu (smoltalk Veri Kümesi ile)

Aşağıdaki tam Python kodu, `smol-ai/smoltalk` veri kümesini kullanarak `SmolLM2-360M` temel modeline SFT uygulamaktadır:

```
from datasets import load_dataset
from transformers import AutoModelForCausalLM, AutoTokenizer
from trl import SFTTrainer, SFTConfig

# 1. Veri Kümesi ve Modelin Yüklenmesi
dataset = load_dataset('smol-ai/smoltalk', 'all', split='train[:5000]')
model_id = 'HuggingFaceTB/SmolLM2-360M'

tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id)

# 2. SFT Yapılandırmasının Tanımlanması
sft_config = SFTConfig(
    output_dir='./sft_smollm2_results',
    max_seq_length=512,
    packing=True,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=2,
    learning_rate=2e-5,
    num_train_epochs=3,
    logging_steps=10,
    save_strategy='epoch',
    evaluation_strategy='no'
)

# 3. SFTTrainer İle Eğitimin Başlatılması
trainer = SFTTrainer(
    model=model,
    args=sft_config,
    train_dataset=dataset,
    processing_class=tokenizer,
)

trainer.train()
print('SFT Eğitimi Başarıyla Tamamlandı!')
```

Kayıp (Loss) Eğrilerinin İzlenmesi ve Teşhis

Eğitim sırasında Eğitim Kaybı (Training Loss) ve Doğrulama Kaybı (Validation Loss) eğrileri yakından takip edilmelidir:

• **Overfitting (Aşırı Öğrenme):** Eğitim kaybı düşmeye devam ederken doğrulama kaybı artmaya başlar. Model veriyi ezberlemiştir. Çözüm: Erken durdurma (early stopping), dropout artırma veya daha fazla veri ekleme.

• **Underfitting (Eksik Öğrenme):** Hem eğitim hem doğrulama kaybı yüksek kalır ve düşmez. Çözüm: Öğrenme oranını artırma, epoch sayısını yükseltme veya model kapasitesini büyütme.

• **İdeal Öğrenme:** Her iki kayıp eğrisi de paralel bir şekilde istikrarlı olarak düşer ve bir noktada plato çizer.

SFT Bölüm Sınavı ve Değerlendirme Soruları

SFT Kavram Kontrolü

Soru 1: Temel bir dil modelini (Base Model) SFT yapmadan doğrudan sohbet asistanı olarak kullanırsak ne olur?

Cevap: Model soruları yanıtlamak yerine sorunun devamını tahmin etmeye çalışır veya metin tamamlama modunda kalır.

Soru 2: SFTTrainer içindeki 'packing=True' parametresi ne işe yarar?

Cevap: Birden fazla kısa eğitim metnini aralarına özel jetonlar koyarak tek bir max_seq_length uzunluğundaki dizide birleştirir, GPU doldurma verimini maksimuma çıkarır.

Soru 3: Overfitting tespit edildiğinde ilk alınması gereken önlem nedir?

Cevap: Eğitim adım sayısını/epoch azaltmak, lora_dropout değerini yükseltmek veya eğitim verisini çeşitlendirmektir.

4. Bölüm: Düşük Rütbeli Uyarlama (LoRA - Low-Rank Adaptation)

Büyük dil modellerinde milyarlarca parametrenin tamamını eğitmek (Full Fine-Tuning) devasa GPU bellek gücü ve depolama alanı gerektirir. **LoRA (Low-Rank Adaptation)**, ana model ağırlıklarını tamamen dondurarak (frozen) katman aralarına eğitilebilir düşük rütbeli matrisler ekleyen bir Parametre Verimli İnce Ayar (PEFT) metodudur.

LoRA Matris Matematiği ve Çalışma Prensibi

LoRA, pre-trained ağırlık matrisi $W_0 \in \mathbb{R}^{d imes k}$ sabit tutulurken, güncellemeyi ifade eden ΔW matrisini iki küçük matrisin çarpımı olarak ayrıştırır: $\Delta W = B \cdot A$. Burada $A \in \mathbb{R}^{r imes k}$ ve $B \in \mathbb{R}^{d imes r}$ olup, $r \ll \min(d, k)$ seçilir. A matrisi Gaussian rastgele değerlerle, B matrisi ise sıfır ile başlatılır. İleri beslemede çıktı: $h = W_0 x + \frac{\alpha}{r} B A x$ olur.

LoraConfig Yapılandırma Parametreleri Tablosu

Parametre	Önerilen Değerler	Açıklama ve Etkisi
r (rank)	8, 16, 32, 64	Düşük rütbe matrisinin boyutudur. Artıkça eğitilebilir parametre sayısı artar.
lora_alpha	16, 32, 64 (Genellikle 2 * r)	LoRA ağırlıklarının ölçekleme katsayısıdır. Öğrenme kararlılığını sağlar.
target_modules	['q_proj', 'v_proj', 'k_proj', 'o_proj']	LoRA matrislerinin ekleneceği Transformer katmanlarıdır.
lora_dropout	0.05 - 0.1	LoRA katmanlarına uygulanan dropout oranı.
bias	'none', 'all', 'lora_only'	Modeldeki sapma (bias) parametrelerinin eğitilip eğitilmeyeceği.
task_type	'CAUSAL_LM'	Modelin görev türünü belirtir (Ör: Causal Language Modeling).

PEFT ve SFTTrainer İle LoRA Eğitimi Kod Örneği

Hugging Face `peft` kütüphanesi ile `LoraConfig` oluşturup doğrudan `SFTTrainer` içerisine aktarmak oldukça basittir:

```
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import LoraConfig, get_peft_model
from trl import SFTTrainer, SFTConfig

model_id = 'HuggingFaceTB/SmolLM2-360M'
model = AutoModelForCausalLM.from_pretrained(model_id)

# 1. LoRA Yapılandırması
peft_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=['q_proj', 'k_proj', 'v_proj', 'o_proj'],
    lora_dropout=0.05,
    bias='none',
    task_type='CAUSAL_LM'
)

# Eğitilebilir parametre oranını yazdırma
peft_model = get_peft_model(model, peft_config)
peft_model.print_trainable_parameters()
# Çıktı: trainable params: 1,843,200 || all params: 362,000,000 || trainable%: 0.509%
```

LoRA Adaptörlerini Ana Modelle Birleştirme (Merging Adapters)

Eğitim tamamlandıktan sonra elde edilen küçük adaptör ağırlıkları, çıkarım (inference) anında gecikme süresi (latency) yaratmamak için `merge_and_unload()` fonksiyonu ile ana model ağırlıklarına kalıcı olarak eklenebilir:

```
from peft import PeftModel
from transformers import AutoModelForCausalLM

base_model = AutoModelForCausalLM.from_pretrained('HuggingFaceTB/SmolLM2-360M')
peft_model = PeftModel.from_pretrained(base_model, './lora_adapter_checkpoint')

# Adaptörleri ana model ağırlıklarına entegre etme
merged_model = peft_model.merge_and_unload()
merged_model.save_pretrained('./smollm2_lora_merged')
```

5. Bölüm: Değerlendirme (Evaluation) Metodolojileri ve Benchmark'lar

İnce ayar yapılmış bir modelin başarısını ölçmek, genel yeteneklerini kaybedip kaybetmediğini (Catastrophic Forgetting) ve hedeflenen görevde ne kadar performans gösterdiğini doğrulamak için sistematik bir **Değerlendirme Süreci** şarttır.

Otomatik Benchmark Standartları

Yapay zeka literatüründe yaygın olarak kabul görmüş otomatik değerlendirme testleri şunlardır:

Benchmark Testi	Ölçülen Yetenek / Alan	Örnek Veri Seti / Format
MMLU	Genel Kültür, Lisans Seviyesi Bilim, Hukuk, Tıp	57 farklı konuda çoktan seçmeli sorular.
GSM8K & MATH	İlkokul ve İleri Düzey Matematiksel Akıl Yürütme	Aşamalı problem çözümü gerektiren matematik soruları.
HumanEval & MBPP	Python Kod Üretme ve Programlama Becerisi	Fonksiyon imza ve docstring'lerinden kod tamamlama (Pass@1).
TruthfulQA	Doğruluk, Yanılsama (Hallucination) Direnci	İnsanların sıkça düştüğü yanlış inançlar ve mitler.
AlpacaEval / MT-Bench	Çok Turlu Talimat Takip ve Konuşma Kalitesi	GPT-4 tabanlı otomatik yanıt puanlaması.

Alternatif Değerlendirme Yöntemleri

Otomatik testlerin yetersiz kaldığı durumlarda kullanılan üç temel alternatif yaklaşım bulunmaktadır:

1. LLM-as-a-Judge (Hakem Olarak LLM): GPT-4 veya Claude 3.5 Sonnet gibi gelişmiş modeller, ince ayar yapılmış modelin ürettiği yanıtları belirlenen değerlendirme ölçeklerine (coherence, helpfulness, safety) göre puanlar.

2. Chatbot Arena (İnsan Tercih Sıralaması): Kullanıcıların iki farklı modelin yanıtını körleme (blind test) yöntemiyle oyladığı ve Elo puanlama sisteminin uygulandığı platformlardır.

3. Özel Alan Test Süitleri (Custom Evaluation Suites): Şirketlerin veya uzmanların kendi kurum içi verileriyle oluşturduğu özel doğrulama testleridir.

Hugging Face lighteval Kütüphanesi İle CLI Değerlendirmesi

Hugging Face `lighteval` aracı, komut satırından modellerin benchmark sonuçlarını hızlıca hesaplamanızı sağlar:

```
# lighteval kütüphanesi ile MMLU değerlendirmesi çalıştırma
lighteval accelerate \
    'pretrained=HuggingFaceTB/SmolLM2-360M' \
    'custom|mmlu|0|0' \
    --output-dir ./lighteval_results
```

Değerlendirme ve SFT Genel Sınavı

Değerlendirme Sınavı ve Cevap Anahtarı

Soru 1: MMLU testi neleri ölçer ve neden önemlidir?
Cevap: 57 farklı akademik disiplinde çoktan seçmeli sorularla modelin genel dünya bilgisini ve akıl yürütme kapasitesini ölçer.

Soru 2: Catastrophic Forgetting (Yıkıcı Unutma) nedir?
Cevap: İnce ayar yapılan bir modelin belirli bir özel görevi öğrenirken, önceden sahip olduğu genel dil yeteneklerini ve bilgisini kaybetmesidir.

Soru 3: LLM-as-a-Judge yönteminin en büyük avantajı nedir?
Cevap: İnsan değerlendirmesine yakın sonuçları çok daha hızlı, ucuz ve ölçeklenebilir biçimde sunmasıdır.

6. Bölüm: Sonuç ve Bütünleşik Uygulama Özeti

Özel bir yapay zeka ve dil modeli çözümü geliştirirken takip edilecek bütünleşik yol haritası 4 temel sütun üzerine inşa edilmiştir:

Başarılı İnce Ayar İçin Yol Haritası

1. Adım - Sohbet Şablonu Seçimi: Hedeflenen model yapısına uygun şablon (ChatML, Llama 3 vb.) seçilir ve `apply_chat_template` entegrasyonu tamamlanır.

2. Adım - Veri Hazırlığı & Paketleme: Kaliteli talimat-yanıt çiftleri oluşturulur, `packing=True` ile GPU verimi artırılır.

3. Adım - SFT + LoRA Eğitimi: Bellek tasarrufu için LoRA adaptörleri ile `SFTTrainer` çalıştırılır; kayıp eğrileri izlenir.

4. Adım - Sistematik Değerlendirme: `lighteval` veya özel test setleri ile model doğrulanır, adaptörler birleştirilerek (`merge_and_unload`) yayına alınır.

— REHBERİN SONU —